UNIVERSIDADE KIMPA VITA

ESCOLA SUPERIOR POLITÉCNICA DO UIGE

CURSO DE ENGENHARIA INFORMATICA

TRABALHO EM GRUPO

DE

LINGUAGEM DE PROGRAMAÇÃO VI

COMPLEXIDADE DOS ALGORITMOS E ARVORES

III °-ANO

PERIÓDO: MANHÃ

O DOCENTE

Msc. MOYO KANIVENGIDIO

UÍGE/ 2026

índice

0. INTRODUÇÃO 1

1. Objetivo Geral 2

1.1. Objetivos Específicos 2

1.2. Justificativa 2

1.3. Metodologia 3

2. Complexidade dos algoritmos 4

2.1. Algoritmos 4

2.2. Comparação de duas soluções para um mesmo problema 5

2.3. Tipos de complexidade 5

2.4. Análise de complexidade de um algoritmo 6

2.5. Notação O 7

2.7. Exercícios: Análise de complexidade 9

3. Árvores 11

3.1. Baseada em Propriedades: 11

3.2. Definição Recursiva: 12

2.3 Conceitos e Definições 14

3.3. Árvore binária 15

3.4. Árvore binária completa 15

3.5. Representação com lista de listas 16

3.6. Representação com referências 19

3.7. Percorrendo uma árvore 20

Referências Bibliográficas 24

Lista dos intergrantes do grupo 25

0. INTRODUÇÃO

A ciência da computação tem evoluído significativamente, tornando os algoritmos e as estruturas de dados elementos fundamentais para o desenvolvimento de soluções computacionais eficientes. Um algoritmo pode ser entendido como um conjunto bem definido de instruções que recebe dados de entrada, processa essas informações e produz resultados desejados. A eficiência desses algoritmos é avaliada por meio da análise de sua complexidade, que mede a quantidade de recursos utilizados durante a execução, como tempo e memória.

A análise da complexidade dos algoritmos permite comparar diferentes soluções para um mesmo problema, identificando qual delas apresenta melhor desempenho à medida que o volume de dados aumenta. Nesse contexto, a notação Big O desempenha um papel essencial ao expressar o comportamento assintótico das funções que representam o tempo de execução dos algoritmos.

Além disso, as árvores representam uma importante estrutura de dados hierárquica utilizada em diversas aplicações computacionais, como organização de arquivos, representação de expressões e implementação de algoritmos de busca. As árvores binárias, em particular, são amplamente utilizadas devido à sua eficiência e simplicidade na manipulação de dados. Dessa forma, compreender os conceitos relacionados à complexidade dos algoritmos e às estruturas de árvores torna-se essencial para o desenvolvimento de sistemas computacionais eficientes e organizados.

1. Objetivo Geral

Compreender os conceitos fundamentais relacionados à complexidade dos algoritmos e às estruturas de árvores, analisando seu funcionamento, classificação e aplicação na resolução eficiente de problemas computacionais.

1.1. Objetivos Específicos

- ✓ Definir o conceito de algoritmo e sua importância na resolução de problemas computacionais.
- ✓ Estudar os diferentes tipos de complexidade de algoritmos, com destaque para a complexidade temporal e espacial.
- ✓ Compreender a análise de complexidade em diferentes cenários, como melhor caso, pior caso e caso médio.
- ✓ Explicar a notação Big O e sua utilização na comparação da eficiência de algoritmos.
- ✓ Identificar os principais conceitos relacionados às árvores, como nós, raiz, folhas, nível e altura.
- ✓ Descrever as características e propriedades das árvores binárias.
- ✓ Estudar as diferentes formas de representação de árvores binárias em programação.
- ✓ Analisar os métodos de percurso em árvores, incluindo pré-ordem, em-ordem e pós-ordem.
- ✓ Aplicar os conhecimentos adquiridos na implementação e manipulação de estruturas de árvores em algoritmos.

1.2. Justificativa

O estudo da complexidade dos algoritmos e das estruturas de árvores é fundamental na área da Engenharia Informática, pois permite compreender como os programas se comportam diante de grandes volumes de dados. Em um cenário tecnológico cada vez mais exigente, a eficiência dos algoritmos torna-se um fator determinante para o bom desempenho dos sistemas computacionais.

Além disso, a análise da complexidade possibilita identificar qual algoritmo é mais adequado para resolver determinado problema, evitando desperdício de tempo e recursos computacionais. Da mesma forma, as árvores, especialmente as árvores binárias, são amplamente utilizadas em diversas aplicações, como organização de dados, sistemas de busca e armazenamento de informações hierárquicas.

Dessa forma, torna-se essencial o estudo desses conceitos, pois contribui para a formação acadêmica e profissional dos estudantes, permitindo o desenvolvimento de soluções computacionais mais eficientes, organizadas e seguras.

1.3. Metodologia

Para a elaboração deste trabalho, foi utilizada uma abordagem baseada na pesquisa bibliográfica e análise de conteúdos relacionados à complexidade dos algoritmos e estruturas de árvores. Inicialmente, realizou-se a leitura e interpretação do material didático fornecido pelo docente, permitindo a compreensão dos conceitos fundamentais sobre algoritmos, análise de complexidade e notação Big O.

Em seguida, foram estudados os diferentes tipos de árvores, com destaque para as árvores binárias, suas propriedades, formas de representação e métodos de percurso. Também foram analisados exemplos práticos e códigos de programação em Python, com o objetivo de compreender a implementação e funcionamento dessas estruturas de dados.

Por fim, as informações coletadas foram organizadas e estruturadas de forma clara e objetiva, permitindo a apresentação dos conceitos e aplicações de maneira compreensível e adequada ao nível acadêmico.

2. Complexidade dos algoritmos

Hoje em dia também existe uma discussão muito recorrente sobre linguagens de programação serem melhores ou mais rápidas que outras. Apesar de um poder ser melhor que outra em situações diferentes, de nada importa se a pessoa que estiver escrevendo o código não tiver uma noção fundamental de algoritmos.
A Complexidade de um Algoritmo consiste na quantidade de "trabalho" necessária para a sua execução, expressa em função das operações fundamentais, as quais variam de acordo com o algoritmo, e em função do volume de dados.

- ✓ Um algoritmo serve para resolver um determinado problema, e todos os problemas têm sempre uma entrada de dados.
- ✓ O tamanho dessa entrada (N) tem geralmente efeito direto no tempo de resposta de um algoritmo.
- ✓ Dependendo do problema a ser resolvido, já existem algoritmos prontos ou que podem ser adaptados

2.1. Algoritmos

No mundo da ciência da computação, algoritmos são processos computacionais bem definidos que podem receber uma entrada, processá-la e produzir um ou mais valores como saída. Alguns exemplos de problemas que envolvem algoritmos comuns são:

- ✓ Calcular a rota mais curta entre duas ruas;
- ✓ Contar o número de amigos em comum em uma rede social;
- ✓ Organizar de maneira eficiente tarefas de acordo com sua prioridade, prazo e duração;
- ✓ Organizar a lista de contatos por ordem alfabética;
- ✓ Encontrar uma mensagem no histórico de conversas.

De forma geral, podemos pensar em algoritmos como uma ferramenta para resolver um problema bem definido. A definição de um problema se baseia em um conjunto de dados que se deseja processar, com suas especificidades, e o resultado que é desejado alcançar. Como podemos perceber, existem diversos problemas que podem ser resolvidos utilizando algoritmos.

Não existe uma "bala de prata" para resolvê-los de maneiras eficiente. Porém existem categorias nas quais os problemas e seus subproblemas podem ser classificados.

Abstraindo o problema, podemos identificar a sua categoria e aplicar técnicas específicas conhecidas para resolvê-los da maneira mais eficiente.

Algumas das principais categorias/técnicas são:

- ✓ Greedy Algorithms;
- ✓ Dynamic Programming;
- ✓ Divide and conquer;
- ✓ Backtracking;
- ✓ Search and Sorting.

2.2. Comparação de duas soluções para um mesmo problema

Tomemos duas soluções para localizar um elemento em um vetor: LIN e BIN. Temos dois computadores diferentes, A e B, sendo A um Core 2 Duo e B um Celeron.

Tamanho (n)	Tempo de execução de LIN em A	Tempo de execução de BIN em B
16	8 ns	100.000 ns
64	32 ns	150.000 ns
256	128 ns	200.000 ns
1024	512 ns	250.000 ns
...	...	...
1.048.576	524.288 ns	500.000 ns
4.194.304	2.097.152 ns	550.000 ns
16.777.216	8.388.608 ns	600.000 ns
...	...	...
$63{,}072 \times 10^{12}$	$31{,}536 \times 10^{12}$ ns ou 1 ano	1.375.000 ns ou cerca de

2.3. Tipos de complexidade

Pode-se falar de dois tipos de complexidade de algoritmos:

- ✓ Complexidade Espacial: Quantidade de recursos utilizados para resolver o problema;
- ✓ Complexidade Temporal: Quantidade de tempo utilizado. Pode ser vista também como o número de instruções necessárias para resolver um determinado problema;

Notas

- ✓ Em ambos os casos, a complexidade é medida de acordo com o tamanho dos dados de entrada (n).

- ✓ Estamos mais interessados em calcular a Complexidade Temporal de um algoritmo!

2.4. Análise de complexidade de um algoritmo

O tempo que um programa consome para uma dada entrada caracterizada por um tamanho n, designa se por f(n) e é dado pelo somatório dos tempos gastos pelas ações elementares que compõem o programa.

	Custo	
int getmax(int*a, int N){		
int i, mx=a[0]	C_0	1
for (i=1;i<N;i++)	C_1	N
If (a[i] >mx)	C_2	N- 1
mx= a[i]	C_3	$\sum_{i=1}^{N-1} n_i$
return mx;	C_4	1
}		

Como exemplo, considera se a seguinte **função em C** que encontra o valor máximo de um array **a** de tamanho N.

$$f(N)=c_0+c_1N+c_2(N-1)+c_3\sum_{i=1}^{N-1} n_i+c_4$$

Algumas observações:

- ❖ Ao arbitrarmos custos $c_0 \ldots c_4$ estamos a tornar a análise independente da plataforma onde a função corre.
- ❖ O termo n_i toma como valores 0 ou 1 e indica se na iteração i, a atribuição mx = a[i] foi ou não executada.
- ❖ Como podemos verificar, f(n) depende não apenas do valor de n mas da própria entrada. Tipicamente distinguem-se 3 casos:
  - ✓ **Pior Caso**, caracterizado por entradas que resultam em maior crescimento do número de operações, conforme aumenta o valor de n;
  - ✓ **Melhor Caso**, caracterizado por entradas que resultam em menor crescimento do número de operações, conforme aumenta o valor de n;
  - ✓ **Caso Médio**, que retrata o comportamento médio do algoritmo, quando se consideram todas as entradas possíveis e as respectivas probabilidades de ocorrência (esperança matemática).

Somente o estudo da complexidade de algoritmos permite a comparação de dois algoritmos equivalentes, isto é, desenvolvidos para resolver o mesmo problema.

Exemplo

Seja o programa acima onde :

$$f(N)=c_0+c_1N+c_2(N-1)+c_3\sum_{i=1}^{N-1}n_i+c_4$$

Como podemos verificar, f(N) depende não apenas do valor de N mas da própria entrada.

1. Melhor Caso:

O melhor caso ocorre quando o array está ordenado por ordem decrescente e o primeiro elemento do array corresponde ao máximo dos elementos do array. Neste caso:

$$\sum_{i=1}^{N-1}n_i=0,$$

pelo que

$$\begin{aligned} f_1(N) &= (c_0+c_4-c_2)+(c_1+c_2)N \\ &= a+bN \end{aligned}$$

Onde a e b são constantes positivas que dependem dos custos c_0, ..., c_4.

2. Pior Caso

O pior caso ocorre quando o array está ordenado por ordem crescente. Neste caso, temos:

$$\begin{aligned} f(N) &= c_0+c_1N+c_2(N-1)+c_3\sum_{i=1}^{N-1}n_i+c_4 \\ &= (c_0+c_4-c_2)+(c_2+c_3)(N-1)+c_1N \\ &= a'+b'N \end{aligned}$$

Onde a e b são constantes positivas que dependem dos custos.

2.5. Notação O

A notação *O* (leia-se ***ó grande***, ou ***big oh***) é utilizada para expressar comparativamente o crescimento assintótico representa a velocidade com que uma função tende ao infinito. No estudo de complexidade de algoritmos é mais interessante saber como se comporta essa função à medida que aumentarmos o valor de n, do que conhecer valores da função correspondentes a particulares valores de n.

f(n) = O(g(n)) se existem n_0 e k com f(n) ≤ k×g(n) para todo n>n_0.

Por exemplo, é mais importante saber que o número de operações executadas num algoritmo dobra se dobrarmos o valor de n, do que saber que para n igual a 100 são executadas 300 operações.

Ao dizermos que uma função de complexidade f(n) é da ordem de n^2, queremos dizer que as duas funções, f(n) e n^2 tendem ao infinito com a mesma velocidade, ou que têm o mesmo comportamento assintótico. Indicamos por f(n) = O(n^2) em matemática essa informação é expressa por um limite:

$$\lim_{n \to \infty} \frac{f(n)}{n^2} = c \ (\ c > 0)$$

Se, por exemplo, outro algoritmo para o mesmo problema tem função de complexidade f_1(n) = O(n), podemos comparar f(n) e f_1(n) e, em consequência, comparar a eficiência dos programas que os implementam. Em um deles, o tempo de execução é linear e no outro, o tempo é quadrático.

Regras

1. f(n) = O(f(n))
2. g= O(f), f= O(h) ⇒ g= O(h)
3. c.O(g(n)= o(c.g(n))= O(g(n))
4. f(n).Og(n))=O(f(n).g(n))
5. O(f) + O(g)= O(f +g)
6. $\lim_{+\infty} \frac{f}{g} \to 0 \quad \Rightarrow O(f+g)=O(f)$

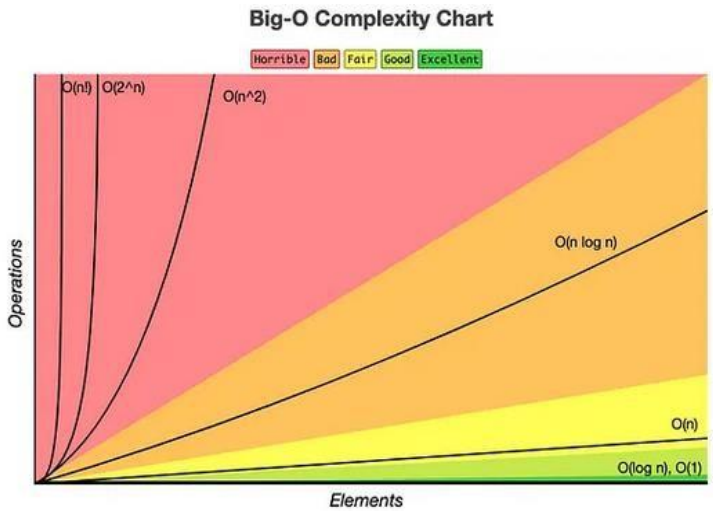
2.6. Convenções para as expressões de O

Existem algumas convenções quanto à expressão de O:

- ✓ É prática comum escrever a expressão de O sem os termos menos significantes.
- ✓ Assim, em vez de O(n^2+ nlogn +n), escrevemos simplesmente O(n^2).
- ✓ É comum desconsiderar os coeficientes constantes. Em lugar de O($3n^2$), escrevemos simplesmente O(n^2). Como caso especial desta regra, se a função é constante, por exemplo O(1024), escrevemos simplesmente O(1).

Algumas expressões de O são tão frequentes que receberam denominações próprias:

Expressão	Nome
O(1)	Constante
O(log n)	Logarítmica
$O(\log_2 n)$	Log quadrado
O(n)	Linear
O(nlog n)	n log n
$O(n^2)$	Quadrática
$O(n^3)$	Cúbica
$O(2^n)$	Exponencial



2.7. Exercícios: Análise de complexidade

1. Calcule a complexidade, no pior caso, do fragmento de código abaixo:

```
int i , j , k ;
 for ( i =0; i < N; i     ++)
        for ( j =0; j < N; j     ++)
                R[ i ] [ j ] = 0;
                for ( k=0; k   <  N; k++)
                        R[ i ] [ j ] += A[ i ] [ k ]        * B[ k ] [ j ]
```

2. **Calcule a complexidade, no pior caso, do fragmento de c´odigo abaixo:**

```
int i , j , k , s ;
for ( i =0; i < N   -1; i  ++)
        for ( j = i +1; j < N; j      ++)
                for ( k=1; k < j ; k    ++)
                        s = 1;
```

3. Calcule a complexidade, no pior caso, do fragmento de código abaixo:

```
int i , j , s ;
s = 0;
for ( i =1; i < N  -1; i  ++)
        for ( j =1; j <  2+N; j ++)
                s = s + 1;
```

4. Obtenha a equação matemática referente à análise do pior e melhor caso do fragmento de código abaixo:

```
        p r i n t f ( ”%d ” , i ) ;
```

for (i = 0; i < N; i ++)

5. Obtenha a equação matemática referente à análise do pior e melhor caso do fragmento de código abaixo:

```
for ( i = 0; i < N; i = i     +2)
        p r i n t f ( ”%d ” , i ) ;
```

6. Obtenha a equação matemática referente à análise do pior e melhor caso do fragmento de código abaixo:

```
for ( i = 0; i < N; i = i     +2)
        p r i n t f ( ”%d ” , i ) ;
        i --;
```

7. Obtenha a equação matemática referente à análise do pior e melhor caso do fragmento de código abaixo:

```
for ( i = 0; i < N; i = i     +2)
        for ( j = N  -i ; j >=0; j   --)
                i f (V[ i ] < V[ j ] )
                        p r i n t f ( ”%d ”   , i )
```

8. Obtenha a equação matemática referente à análise do pior e melhor caso do fragmento de código abaixo:

```
for ( i = 1; i <= N; i    =2 + i )
    p r i n t f ( ”%d ” , i ) ;
```

3. Árvores

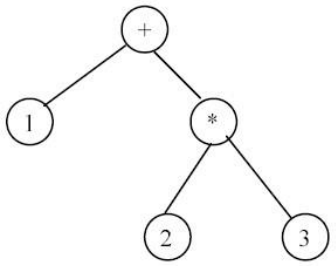
Neste capítulo, exploraremos árvores, uma estrutura de dados comumente utilizada em diversas aplicações. Intuitivamente, uma árvore representa uma organização da informação onde os dados estão em uma relação hierárquica. Na verdade, o uso de árvores para representar relações é anterior ao advento dos computadores.

Na ciência da computação, as árvores são utilizadas para representar decisões, estruturas de linguagem formal e hierarquias entre elementos, bancos de dados, sistemas operacionais e algoritmos de busca.

Por exemplo, uma árvore é uma representação natural da decomposição de uma frase em português em seus elementos constituintes, como sujeito, verbo, complemento direto, etc. Nos computadores, os arquivos são organizados em uma estrutura de árvore: arquivos são armazenados em diretórios (pastas), que por sua vez podem estar contidos em outras pastas.

No contexto da programação, uma árvore é uma estrutura de dados que reflete a topologia hierárquica, onde os dados são organizados de forma que um conjunto de dados está subordinado hierarquicamente a outro.

Expressões algébricas, ou mais geralmente expressões de uma linguagem formal, são representadas como árvores. Por exemplo, a expressão "1 + 2 * 3" pode ser representada por uma árvore.



3.1. Baseada em Propriedades:

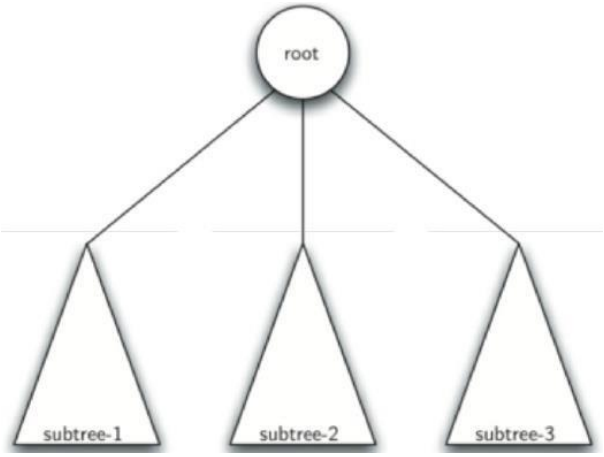
Uma árvore é um tipo específico de grafo, onde, informalmente, um grafo consiste em uma coleção não vazia de nós (ou vértices) e arcos que conectam dois nós. De forma mais formal, uma árvore pode ser caracterizada por um par ordenado (N, A), onde N é um conjunto não vazio de vértices e A é um conjunto não vazio de arcos, satisfazendo um conjunto de propriedades. Uma árvore possui as seguintes propriedades:

- ✓ Toda árvore tem um nó designado como raiz, a partir do qual a busca, inserção e remoção de elementos devem iniciar.
- ✓ Todo nó, exceto a raiz, está conectado por uma aresta a exatamente um nó pai.
- ✓ Existe um único caminho da raiz a qualquer nó arbitrário da árvore.

- ✓ Se cada nó da árvore possui no máximo dois nós filhos, a árvore é chamada de árvore binária.

3.2. Definição Recursiva:

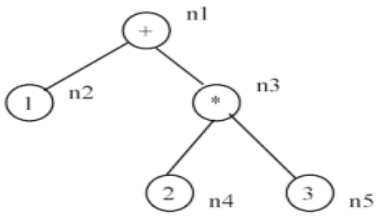
Uma árvore é vazia ou consiste de uma raiz com zero ou mais subárvores, cada uma sendo uma árvore. A raiz de cada subárvore está conectada à raiz da árvore pai por uma aresta. Essas definições nos permitem compreender as propriedades fundamentais das árvores e sua estrutura recursiva.



Na definição recursiva, sabemos que a árvore mencionada possui pelo menos 4 nós, já que cada triângulo representando uma subárvore deve ter uma raiz. No entanto, pode ter muito mais nós, pois não conhecemos a estrutura interna de cada subárvore.

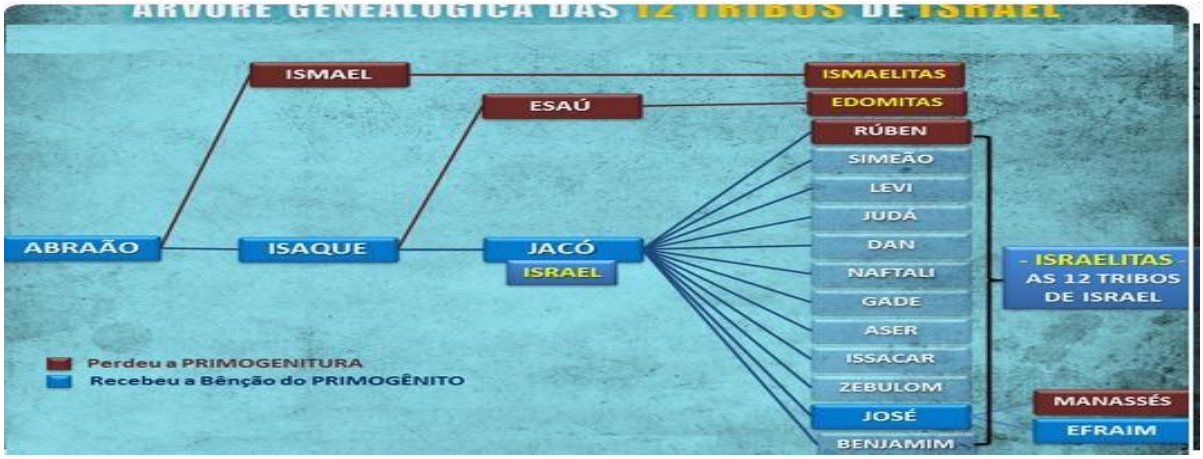
Existem basicamente duas formas de implementar árvores binárias em Python: utilizando uma lista de sublistas ou utilizando encadeamento lógico, similar ao utilizado em listas encadeadas.

Antes de proceder com a implementação de uma árvore binária em Python, vamos listar os principais métodos que essa estrutura de dados deve suportar:



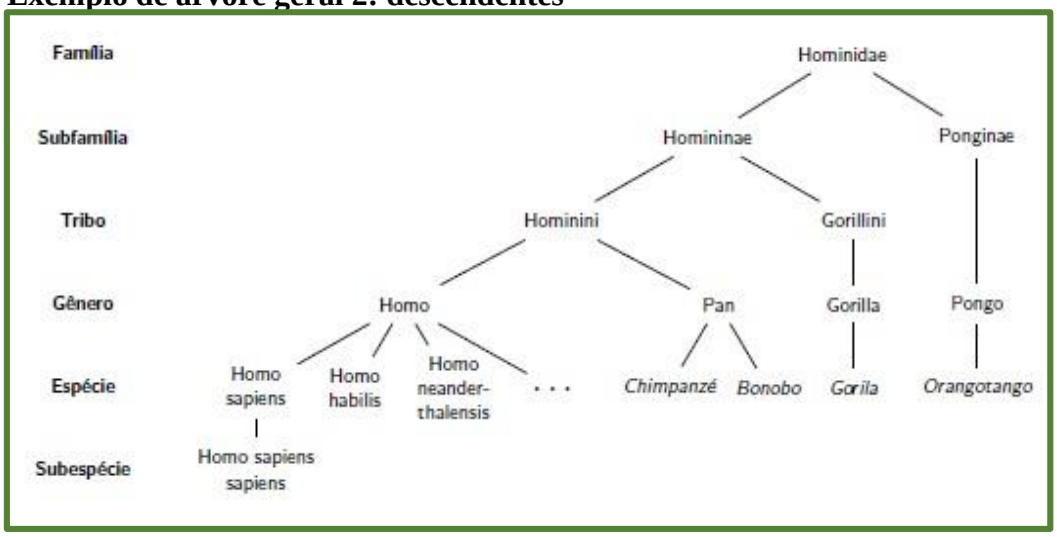
por (N, A) *onde* $N = \{n1, n2, n3, n4, n5\}$ *e* $A = \{(n1, n2), (n1, n3), (n3, n4), (n3, n5)\}$

Exemplo de árvore geral 1: descendentes

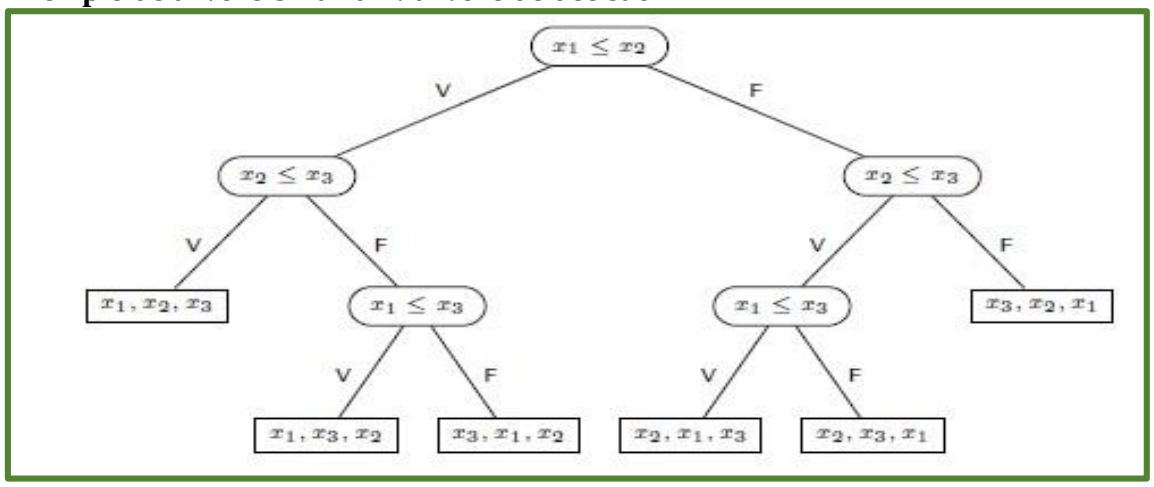


Cada elemento pode ter um número variável de sucessores: árvore geral.

Exemplo de árvore geral 2: descendentes

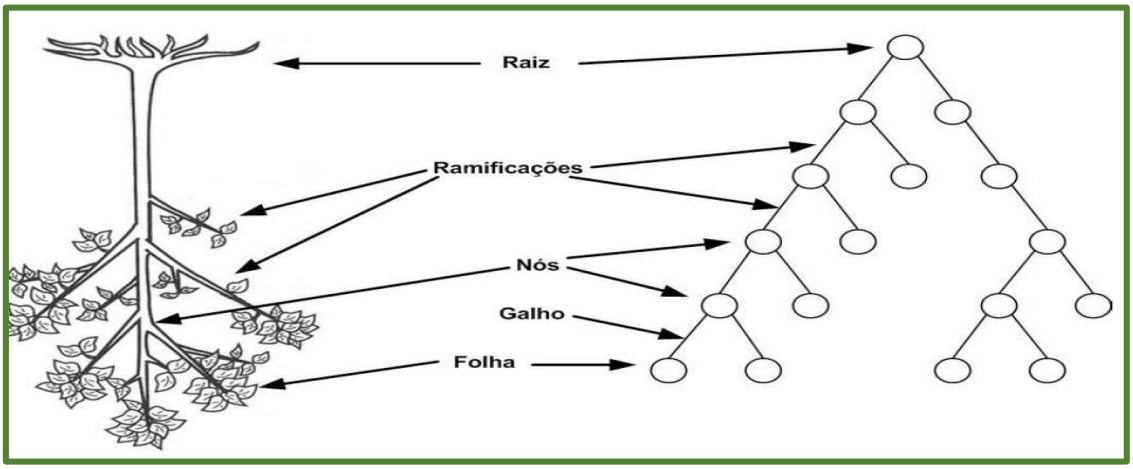


Exemplo de árvore binária 2: árvore de decisão



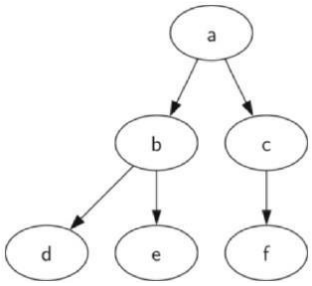
- ✓ A árvore representa as decisões tomadas por um algoritmo de ordenação usando operações de comparação; V: verdadeiro, F: falso.
- ✓ Devido à natureza das comparações, a árvore é binária.

Exemplo de árvore geral



2.3 Conceitos e Definições

A figura a seguir ilustra uma simples arvore composta por 6 nos, sendo que a e a raiz e d, e, f são as folhas.



A seguir apresentamos os conceitos e definições sobre árvores:

- ✓ **Nós:** São os elementos fundamentais de uma árvore, onde as informações são armazenadas.
- ✓ **Arestas:** São as conexões entre os diferentes nós da árvore.
- ✓ **Raiz:** O nó raiz é o ponto de partida da árvore e não possui um nó antecessor; ele é conectado apenas por arestas de saída.
- ✓ **Caminho:** É uma sequência ordenada de nós conectados por arestas. Por exemplo, **a**
- ✓ **→ b → e.**
- ✓ **Nós Filhos:** São todos os nós que possuem uma aresta de entrada proveniente do mesmo nó. Por exemplo, os filhos do nó **b** são **d** e **e**.
- ✓ **Nó Pai:** Um nó é pai de todos os nós conectados a ele por meio de arestas de saída. Por exemplo, o nó **b** é pai dos nós **d** e **e**.
- ✓ **Nós Irmãos:** São nós que possuem o mesmo pai. Por exemplo, os nós **b** e **c** são irmãos.

- ✓ **Subárvore:** É um conjunto de nós e arestas composto por um pai e todos os seus descendentes. Por exemplo, a subárvore da direita em relação à raiz, composta por **b, d** e **e**, é um exemplo.
- ✓ **Nó Folha:** É todo nó que não possui filhos. Neste caso, os nós **d, e** e **f** são folhas.
- ✓ **Nível de um Nó:** É o número de arestas no caminho da raiz até o nó em questão. Os nós **b** e **c** estão no nível 1 e os nós **d, e** e **f** estão no nível 2. A raiz, por definição, pertence ao nível zero.
- ✓ **Altura:** É o maior nível de um nó pertencente à árvore. Neste exemplo, a altura da árvore é 2.

3.3. Árvore binária

Uma árvore binária é um conjunto finito de nós (elementos) que:

- ✓ ou é vazio (árvore binária vazia)
- ✓ ou contém um nó especial denominado raiz da árvore e o resto do conjunto está particionado em duas árvores binárias disjuntas (possivelmente vazias), denominadas subárvore esquerda e subárvore direita.

A árvore pode não ter nenhum elemento (árvore vazia). A definição é recursiva e, devido a isso, muitas operações sobre árvores binárias utilizam recursão.

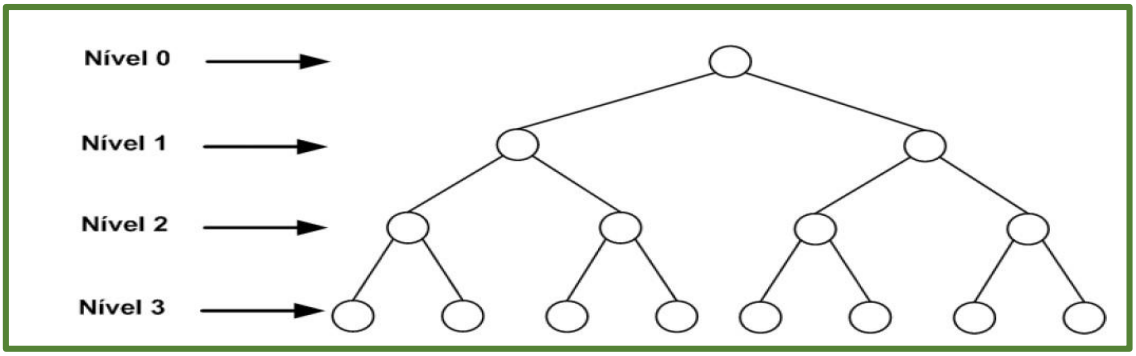
Uma árvore binária é uma estrutura hierárquica composta por nós, onde cada nó pode ter até dois filhos: um filho esquerdo e um filho direito. A partir de um nó, é possível percorrer a árvore seguindo os ramos para acessar outros nós.

As árvores onde cada nó que não seja folha numa árvore binária tem ***subárvores esquerda*** e ***direita*** não vazias são conhecidas como árvores estritamente binárias. Uma árvore estritamente binária com n folhas tem $2^n - 1$ nós.

3.4. Árvore binária completa

Uma árvore completa é uma árvore *estritamente* binária na qual todas as folhas estão no mesmo nível k. Sendo k a profundidade da árvore, o número total de nós é $2^{k+1}-1$ e o número total de folhas é 2^k.

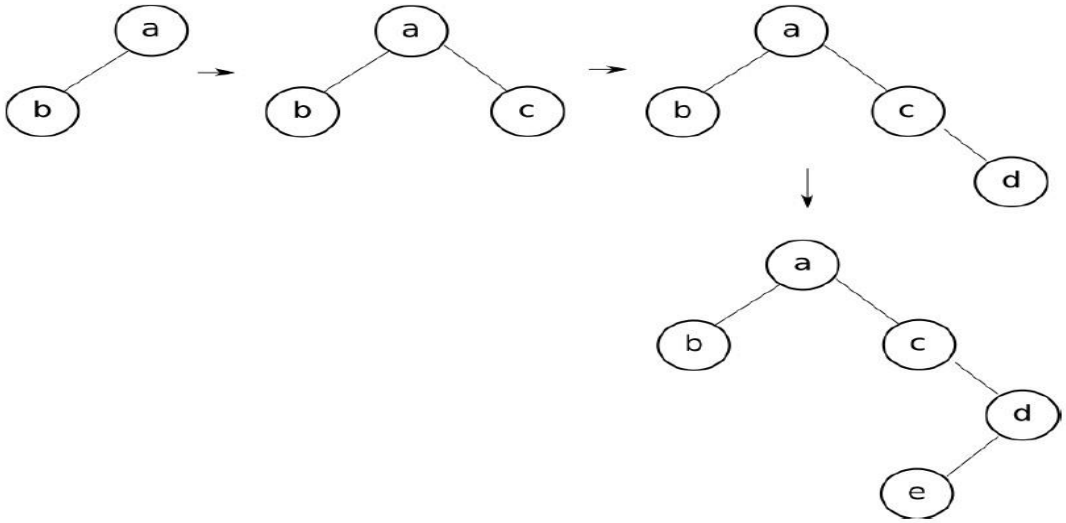
Embora uma árvore binária completa possua muitos nós (o máximo para cada profundidade), a distância da raiz a uma folha qualquer é relativamente pequena.



3.5. Representação com lista de listas

Em uma representação de lista de sublistas, o valor armazenado na raiz e sempre o primeiro elemento da lista. O segundo elemento da lista será a sublista que representa a subarvore a esquerda.

Analogamente, o terceiro elemento da lista será a sublista que representa a subarvore a direita.



Para ilustrar como fica a representação da arvore mostrada na figura em acima, apresentamos o código a seguir.

 # Representação de arvore como lista de sublistas

```
my_tree = ['a', # raiz
                ['b', # subarvore esquerda
                        ['d', [ ] [ ]],
                        ['e'. [ ], [ ]]
                ],
                ['c', # subarvore direita
                        ['f', [ ], [ ]],
                        [ ]
                ]
        ]
```

```
print(my_tree)
print('Subarvore esquerda = ', my_tree[1])
print('Raiz = ', my_tree[    0])
print('Subarvore direita = ', my_tree[2])
```

Uma propriedade muito interessante desta representação e que a estrutura de uma lista representando uma subarvore possui a mesma estrutura de uma arvore (lista), de modo que define uma representação recursiva.

A implementação a seguir ilustra um conjunto de funções que nos permite manipular uma lista de listas de modo a simular o comportamento de uma arvore binaria. Vamos listar os principais métodos que essa estrutura de dados deve suportar.

Método	Descrição
BinaryTree()	Cria uma nova instância da árvore binária.
get_left_child()	Retorna a subárvore à esquerda do nó atual.
get_right_child()	Retorna a subárvore à direita do nó atual.
set_root_val(val)	Armazena um valor no nó atual.
get_root_val()	Retorna o valor armazenado no nó atual.
insert_left(val)	Cria uma nova árvore binária à esquerda do nó atual.
insert_right(val)	Cria uma nova árvore binária à direita do nó atual.

Cria arvore binaria com raiz r def binary_tree(r):
return [r, [ ], [ ]]

# Insere novo ramo a esquerda da raiz def insert_left(root, new_branch): # Analisa a subarvore a esquerda
t = root.pop(1)
# Se a subarvore a esquerda não e vazia if len(t) > 1:
# Insere na posição 1 da raiz (esquerda)

Novo ramo será a raiz da subarvore a esquerda # Adiciona t na esquerda do novo ramo
root.insert(1, [new_branch, t, [ ]]) else:
# Se t for vazia, não ha subarvore a esquerda
root.insert(1, [new_branch, [ ], [ ]])
return root

Insere novo ramo a direita da raiz def insert_right(root, new_branch): # Analisa a subarvore a direita t = root.pop(2)
Se a subarvore a direita não e vazia if len(t) > 1:
Insere na posição 2 da raiz (direita)
Novo ramo será a raiz da subarvore a direita # Adiciona t na direita do novo ramo
root.insert(2, [new_branch, [], t]) else:
Se t for vazia, não ha subarvore a direita root.insert(2, [new_branch, [ ], [ ]])
return root

def get_root_val(root):
return root[0]

def set_root_val(root, new_val):
root[0] = new_val

def get_left_child(root): return root[1]
def get_right_child(root):
return root[2]

if __name__ == '__main__': # Cria arvore binaria r = binary_tree(3)
# Adiciona subarvore a esquerda insert_left(r, 4)
# Adiciona subarvore a esquerda insert_left(r, 5)
# Adiciona subarvore a direita insert_right(r, 6)
# Adiciona subarvore a direita
insert_right(r, 7) print(r)
# Obtem subarvore a esquerda da raiz
l = get_left_child(r)
print(l)
# Muda a raiz da subarvore a esquerda
set_root_val(l, 9)
print(r)
# Insere a esquerda da subarvore a esquerda insert_left(l, 11) print(r)

Exemplo: Considere o seguinte código em Python.
x = binary_tree('a') insert_left(x,'b') insert_right(x,'c')
insert_right(get_right_child(x), 'd')
insert_left(get_right_child(get_right_child(x)), 'e')

3.6. Representação com referências

A representação de árvores binárias por meio de listas de listas não é muito intuitiva, especialmente quando o número de nós na árvore aumenta. O aumento no número de sublistas torna propenso a erros e equívocos durante a manipulação da estrutura. Portanto, exploraremos agora uma maneira de representar árvores binárias usando referências e um encadeamento lógico semelhante ao das listas duplamente encadeadas.

Nesse método, cada nó possui duas referências: uma para o nó filho à esquerda e outra para o nó filho à direita. O código em Python a seguir mostra uma implementação da classe BinaryTree utilizando referências.

```
# Define a classe arvore binaria utilizando referencias class BinaryTree: # Construtor
def __init__(self, valor):
self.key = valor self.left_child = None
self.right_child = None

# Insere nó a esquerda def insert_left(self, valor):
# Se nó corrente não tem filho a esquerda, OK if self.left_child == None:
self.left_child = BinaryTree(valor) else:
# Se tem filho a esquerda, pendura subarvore
# a esquerda do nó corrente na esquerda do
# novo nó recém criado temp = BinaryTree(valor) temp.left_child = self.left_child
self.left_child = temp

# Insere nó a direita def insert_right(self, valor):
# Se no corrente nao tem filho a direita, OK if self.right_child == None:
self.right_child = BinaryTree(valor) else:
# Se tem filho a direita, pendura subarvore
# a direita do nó corrente na direita do
# novo nó recem criado temp = BinaryTree(valor) temp.right_child = self.right_child
self.right_child = temp

# Obtem filho a direita def get_right_child(self):
return self.right_child

# Obtem filho a esquerda def get_left_child(self):
return self.left_child # Atualiza valor do nó corrente def set_root_val(self, valor):
self.key = valor # Obtem valor do nó corrente def get_root_val(self):
return self.key

if __name__ == '__main__': # Cria no com valor 'a' r = BinaryTree('a')
print(r.get_root_val()) print(r.get_left_child())
print(r.get_right_child())

# Insere no com valor 'b' a esquerda da raiz r.insert_left('b')
print(r.get_left_child().get_root_val())
```

```
# Insere no com valor 'c' na direita da raiz r.insert_right('c')
print(r.get_right_child().get_root_val())

# Insere no com valor 'd' a esquerda no filho a esquerda da raiz
r.get_left_child().insert_left('d') print(r.get_left_child().get_left_child().get_root_val())

# Insere no com valor 'e' a direita no filho a esquerda da raiz
r.get_left_child().insert_right('e')
print(r.get_left_child().get_right_child().get_root_val())
```

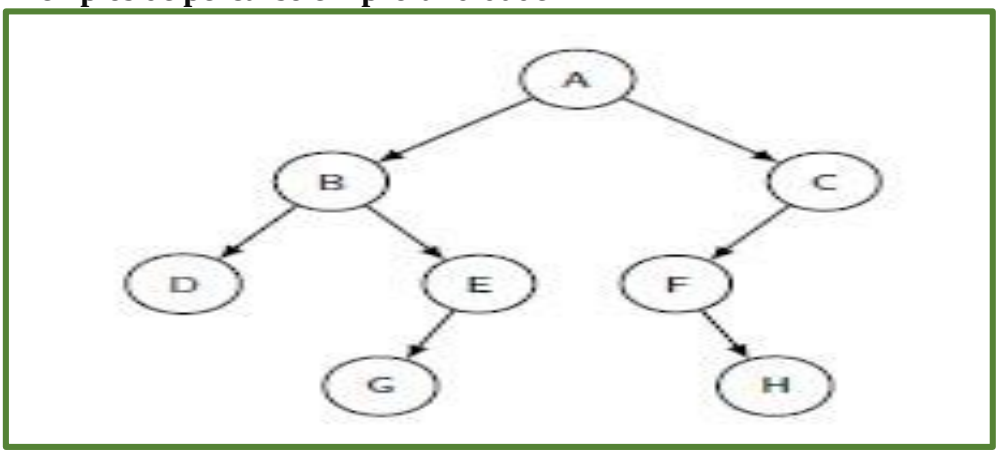
3.7. Percorrendo uma árvore

Uma operação comum é percorrer uma árvore binária, o que consiste em visitar todos os nós desta árvore segundo algum critério. Esse percurso, também chamado de *travessia* da árvore, pode ser feito de três formas:

- ✓ Pré-ordem ou profundidade - os filhos de um nó são processados após o nó.
- ✓ Visitar a raiz
- ✓ Percorrer a subárvore esquerda em pré-ordem
- ✓ Percorrer a subárvore direita em pré-ordem Pós-ordem - os filhos são processados antes do nó.
- ✓ Percorrer a subárvore esquerda em pós-ordem
- ✓ Percorrer a subárvore direita em pós-ordem
- ✓ Visitar a raiz

- ✓ Em-ordem ou simétrica - em que se processa o filho à esquerda, o nó, e finalmente o filho à direita.
- ✓ Percorrer a subárvore esquerda em inordem
- ✓ Visitar a raiz
- ✓ Percorrer a subárvore direita em inordem

Exemplos de percurso em profundidade



Pré-ordem: A,B,D,E,G,C,F,H
Pós-ordem: D,G,E,B,H,F,C,A
Inordem: D,B,G,E,A,F,H,C

Assim, o codigo em Python para a classe BinaryTree completa e listado a seguir.

# Define a classe arvore binaria utilizando referencias class BinaryTree: # Construtor
def __init__(self, valor):
self.key = valor self.left_child = None
self.right_child = None

# Insere no a esquerda def insert_left(self, valor):
# Se no corrente não tem filho a esquerda, OK if self.left_child == None:
self.left_child = BinaryTree(valor)
else:
# Se tem filho a esquerda, pendura subarvore
# a esquerda do nó corrente na esquerda do
# novo nó recém criado temp = BinaryTree(valor) temp.left_child = self.left_child
self.left_child = temp

# Insere nó a direita def insert_right(self, valor):
# Se no corrente não tem filho a direita, OK if self.right_child == None:
self.right_child = BinaryTree(valor)
else:
# Se tem filho a direita, pendura subarvore
# a direita do nó corrente na direita do
# novo nó recém criado temp = BinaryTree(valor) temp.right_child = self.right_child
self.right_child = temp

Obtem filho a direita def get_right_child(self):
return self.right_child

Obtem filho a esquerda def get_left_child(self):
return self.left_child

# Atualiza valor do no corrente def set_root_val(self, valor):
self.key = valor

Obtem valor do no corrente def get_root_val(self):
return self.key

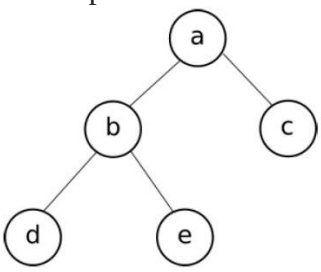
# Percorre uma arvore binaria em Preorder def preorder(self): # Imprime valor da raiz
print(self.key) # Visita subarvore a esquerda if self.left_child: self.left_child.preorder()
# Visita subarvore a direita if self.right_child:
self.right_child.preorder()

Percorre uma arvore binaria em Inorder def inorder(self): # Visita subarvore a
esquerda if self.left_child:
self.left_child.inorder() # Imprime valor da raiz print(self.key) # Visita subarvore a
direita if self.right_child:
self.right_child.inorder()

Percorre uma arvore binaria em Postorder def postorder(self): # Visita subarvore a
esquerda if self.left_child:

```
self.left_child.postorder()

# Visita subarvore a direita if self.right_child:
self.right_child.postorder()

# Imprime valor da raiz
print(self.key)


if __name__ == '__main__': # Cria no com valor 'a' r = BinaryTree('a')
print(r.get_root_val()) print(r.get_left_child()) print(r.get_right_child())

# Insere no com valor 'b' a esquerda da raiz r.insert_left('b')
print(r.get_left_child().get_root_val())

# Insere no com valor 'c' na direita da raiz r.insert_right('c')
print(r.get_right_child().get_root_val())

# Insere no com valor 'd' a esquerda no filho a esquerda da raiz
r.get_left_child().insert_left('d') print(r.get_left_child().get_left_child().get_root_val())

# Insere no com valor 'e' a direita no filho a esquerda da raiz
r.get_left_child().insert_right('e')
print(r.get_left_child().get_right_child().get_root_val())

# Percurso preorder print('Preorder') r.preorder() # Percurso inorder print('Inorder')
r.inorder()
# Percurso postorder print('Postorder') r.postorder()
```

Note que a arvore criada no exemplo acima e ilustrada pela figura a seguir.



Como esperado, os percursos obtidos com os metodos preorder, inorder e postorder foram:

preorder: a,
b, d, e, c
inorder: d, b,
e, a, c
postorder: d, e, b, c, a

Conclusão

Conclui-se que a complexidade dos algoritmos desempenha um papel essencial na avaliação da eficiência de soluções computacionais, permitindo identificar qual algoritmo apresenta melhor desempenho em diferentes situações. O estudo da notação Big O mostrou-se fundamental para compreender o crescimento do tempo de execução dos algoritmos em função do tamanho dos dados de entrada.

Além disso, verificou-se que as árvores são estruturas de dados extremamente importantes na organização e manipulação de informações hierárquicas. As árvores binárias, em particular, destacam-se por sua eficiência e facilidade de implementação, sendo amplamente utilizadas em diversas aplicações computacionais.

Portanto, o conhecimento sobre complexidade de algoritmos e estruturas de árvores é indispensável para o desenvolvimento de sistemas computacionais eficientes e organizados, contribuindo significativamente para a formação de profissionais capacitados na área de Engenharia Informática.

Referências Bibliográficas

AARON M.T.; LANGSAM, Y.; MOSHA, J.A. Estruturas de Dados Usando C. Pearson Education, 2005.

CORMEN T.H.; LEISERSON C.E.; RIVEST, R.L. Algoritmos -Teoria e Prática. Elsevier, 2012.

F. Lorenzi, P. N. de Mattos, and T. P. de Carvalho. Estrutura de dados.Thomson, first edition, 2007 .

SEDGEWICK, R. Algorithms in C. 3ª ed. Pearson Education, 1998.

ZIVIANI, N. Projeto de Algoritmos - com implementação em Pascal e C. Cengage Learning, 1992.

LEE, K. D., HUBBARD, S., Data structures and algorithms with Python, Undergraduate Topics in Computer Science, Springer, 2015.

Lista dos intergrantes do grupo

1. António Almeida K. Nzinga
2. Filipe João Miguel
3. Filipe Pedro Muanza
4. Guiola André
5. Kiasissua simão Pedro
6. Luís Mucuzo António
7. Maria Tumba D. João
8. Nsimba Alberto Samuel
9. Suzana Diasivi Nicolau